SOURCE CODE SECURITY AUDIT REPORT

CRITICAL SEVERITY - IMMEDIATE REMEDIATION REQUIRED

Target Application:

OWASP Juice Shop

OWASP Juice Shop (benchmark)

Report Date	November 11, 2026
Testing Type	Source-code security audit (white-box): AI-assisted manual review, SAST, secrets, dependency and IaC analysis
Target	juice-shop
Risk Rating	**CRITICAL**
Total Findings	**28 Confirmed | 1 Requires Verification**
Classification	**CONFIDENTIAL - Authorized Personnel Only**
Prepared by	Vijaya Kumar L
Reviewed by	Vijaya Kumar L

1. Executive Summary

OWASP Juice Shop is an intentionally vulnerable application; this source-only audit confirmed 28 distinct vulnerabilities spanning SQL and NoSQL injection, authentication and JWT key management, broken access control, server-side code execution (XXE, eval/RCE, SSTI), SSRF, multiple XSS sinks, sensitive-file exposure and vulnerable dependencies. One further issue (unsigned-JWT acceptance) requires a live check of an end-of-life dependency.

1.1 Vulnerability Summary

#	Vulnerability	Severity	Category	CVSS	Status
F-05	JWT forgery via hardcoded RSA private key and end-of-life jsonwebtoken	CRITICAL	Broken Authentication / JWT forgery / Vulnerable component	9.3	Open
F-18	Mass assignment lets a user self-register as admin	CRITICAL	Improper Input Validation / Broken Access Control	9.3	Open
F-01	SQL injection in login allows authentication bypass and full Users table disclosure	HIGH	SQL Injection / Authentication bypass	8.8	Open
F-02	SQL injection in product search enables UNION-based database exfiltration	HIGH	SQL Injection	8.7	Open
F-03	NoSQL injection via MongoDB $where in product reviews and order tracking	HIGH	NoSQL Injection	8.8	Open
F-04	NoSQL injection and missing authorization let any user overwrite all product reviews	HIGH	NoSQL Injection / Broken Access Control	7.1	Open
F-09	XML external entity (XXE) processing enables file disclosure and denial of service	HIGH	XXE	7.2	Open
F-10	Denial of service via unbounded YAML deserialization of uploaded files	HIGH	Insecure Deserialization / Resource exhaustion	7.1	Open
F-11	Server-side code execution via eval of B2B order data	HIGH	Insecure Deserialization / RCE	8.7	Open

OWASP Juice Shop (benchmark)

#	Vulnerability	Severity	Category	CVSS	Status
F-12	Server-side template injection and unsafe CSP via the username field	HIGH	Server-Side Template Injection / XSS	8.7	Open
F-19	Product records can be modified because the PUT route is left unprotected	HIGH	Broken Access Control	7.1	Open
F-23	Passkey registration overwrites existing credentials and session is issued regardless of verification	HIGH	Broken Access Control / Broken Authentication	8.6	Open
F-24	Password hash and other sensitive fields leak via the currentUser fields parameter	HIGH	Sensitive Data Exposure / Excessive data exposure	7.1	Open
F-26	Password can be changed without knowing the current password	HIGH	Broken Authentication / CSRF	7.1	Open
F-07	Passwords stored as unsalted MD5 hashes	MEDIUM	Cryptographic failure / weak password storage	6.9	Open
F-08	Open redirect: allowlist uses substring match on the full URL	MEDIUM	Unvalidated Redirect	5.1	Open
F-13	Server-side request forgery via profile image URL fetch	MEDIUM	SSRF	5.3	Open
F-14	DOM XSS via DomSanitizer.bypassSecurityTrustHtml on untrusted data	MEDIUM	Cross-Site Scripting (XSS)	5.1	Open
F-15	Directory listing and public download of the FTP folder expose forgotten backups	MEDIUM	Sensitive Data Exposure / Security Misconfiguration	6.9	Open
F-16	Server access logs are exposed through a browsable /support/logs directory	MEDIUM	Sensitive Data Exposure / Observability failure	6.9	Open
F-17	Prometheus metrics endpoint is exposed without authentication	MEDIUM	Information Disclosure / Observability misconfiguration	6.9	Open

#	Vulnerability	Severity	Category	CVSS	Status
F-20	Sensitive routes are gated only by client-side guards (admin, scoreboard, web3, token sale)	MEDIUM	Broken Access Control / Security through obscurity	6.9	Open
F-21	Chatbot coupon tool can be driven by prompt injection to mint discounts	MEDIUM	LLM Prompt Injection	5.3	Open
F-22	Path traversal via poison null byte in the FTP file server	MEDIUM	Path Traversal / Improper Input Validation	6.9	Open
F-25	Password-reset rate limit is bypassable via the X-Forwarded-For header	MEDIUM	Broken Anti-Automation	6.9	Open
F-27	Known-vulnerable and end-of-life dependencies are pinned (sanitize-html 1.4.2, jsonwebtoken 0.4.0, express-jwt 0.1.3)	MEDIUM	Vulnerable and Outdated Components	6.9	Open
F-28	Typosquatted frontend dependency present (ngy-cookie)	MEDIUM	Supply Chain / Vulnerable Components	6.9	Open
F-29	Stored XSS via video subtitle template injection	MEDIUM	Cross-Site Scripting (XSS)	5.1	Open
V-06	Unsigned (alg=none) JWTs are accepted because the token is decoded without verifying the signature	UNVERIFIED	Broken Authentication / JWT / Vulnerable component	TBD	Needs Verification

1.2 Severity Distribution

CRITICAL	HIGH	MEDIUM	LOW / UNVERIFIED
2	12	14	1

2. Assessment Scope & Methodology

2.1 Scope

- Target: juice-shop (source code, commit 266448a)
- In-scope paths: app.ts, server.ts, lib, routes, models, config, data/datacache.ts, data/datacreator.ts, data/mongodb.ts, data/staticData.ts, data/types.ts, data/static/challenges.yml,

data/static/contractABIs.ts, data/static/deliveries.yml, data/static/i18n, data/static/locales.json, data/static/securityQuestions.yml, data/static/tags.yml, data/static/users.yml, data/static/web3-snippets, frontend/src/app, frontend/src/hacking-instructor, frontend/src/main.ts, frontend/src/environments, frontend/src/mocks, frontend/src/index.html, frontend/src/assets/i18n, frontend/package.json, ftp, encryptionkeys, views, i18n, swagger.yml, package.json, Dockerfile, infrastructure, terraform, monitoring

- Languages: TypeScript, Terraform, Solidity
- Frameworks and services detected: Angular, Docker, Docker Compose, Express, Git history present, Multer, Sequelize, Socket.IO, Terraform, Vercel AI SDK, jsonwebtoken
- Profile: standard (3 independent verification vote(s) per candidate)
- Audit period: November 11, 2026

2.2 Methodology

The audit followed a six-phase, evidence-gated workflow aligned with the OWASP Testing Guide, OWASP Top 10:2025, OWASP API Security Top 10:2023 and, where relevant, the OWASP Top 10 for LLM Applications 2025:

- Phase 0 - Scope and safety gate: read-only, source-only, no target code executed, no live systems contacted
- Phase 1 - Recon: architecture, principals, trust boundaries, entry points, sensitive-data inventory, coverage ledger
- Phase 2 - Automated scanning: SAST, secrets (files and Git history), dependency CVEs, IaC and container misconfiguration
- Phase 3 - AI-assisted manual hunting by attack class, with coverage critics and variant analysis
- Phase 4 - Independent verification: each candidate re-checked by verifiers that did not find it; every cited file and line machine-checked
- Phase 5 - Reporting from validated records only, with recommendations (no code changed)

2.3 Tools

Tool	Version	Purpose	Status
semgrep	1.184.0	code patterns (SAST)	Ran (JSON->SARIF)
gitleaks	8.30.1	secrets in files and Git history	Ran
trivy	Version: 0.76.1	dependency CVEs, IaC/Dockerfile misconfiguration, secrets	Ran
osv-scanner	osv-scanner version: 2.7.0	known vulnerabilities in lockfiles	Skipped (no lockfile)

2.4 Cost and speed

Engine phases only. 'Engine I/O (chars/4)' counts the characters the tool itself read, divided by 4 - it is not LLM billing. AI token use isn't measured by the engine; check /cost in Claude Code after a run.

Phase	Time (s)	Files read	Engine I/O (chars/4)
validate	0.21	50	~127023
Total (engine)	**0.21**	**50**	**~127023**

3. Detailed Findings

Each finding below passed the evidence gate: a named attacker, the input they control, the rule that should stop them, a source trace with file and line, the missing check, and the concrete result. Live secrets are masked. Exploit payloads are intentionally not included; the engineering team should reproduce each issue in a safe environment as part of remediation.

F-05 JWT forgery via hardcoded RSA private key and end-of-life jsonwebtoken [CRITICAL]

Location	lib/insecurity.ts:54
In plain words	What this is: The secret key that signs login tokens is written directly in the source code. Problem: Anyone who can read the code (it is public) can forge a valid token for any account, including admin. If we skip it: Attackers mint admin sessions at will; authentication is effectively meaningless.
Description	The RSA private key is committed in lib/insecurity.ts, so an attacker signs a token for any user (including admin) and is fully authenticated
Business Impact	Total authentication bypass and privilege escalation to admin by forging session tokens.
Root Cause	The RSA signing key is a hardcoded literal in a committed, public file, and the JWT library is end-of-life.

Evidence (source trace)	`1. lib/insecurity.ts:54  (signs with privateKey, a hardcoded RSA key defined at lib/insecurity.ts:21)`
	`    export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256' })`
	`2. package.json:123  (end-of-life version with algorithm-confusion weaknesses)`
	`    "jsonwebtoken": "0.4.0",`

Recommendation	Load the signing key from a secret manager or environment variable, rotate the exposed key immediately, and upgrade jsonwebtoken to a supported release. Files to change: lib/insecurity.ts, package.json
Fix example	`const privateKey = process.env.JWT_PRIVATE_KEY // loaded from a secret store, never committed`
Production risk of the fix	All existing tokens become invalid on key rotation, forcing re-login; schedule during a maintenance window.

Regression test	Add a test that signs a token with the old committed key and asserts the server rejects it after rotation.
Classification	CWE-321, CWE-347 | A02:2021 Cryptographic Failures, A07:2021 Identification and Authentication Failures | CVSS 9.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-18 Mass assignment lets a user self-register as admin [CRITICAL]

Location	server.ts:550
In plain words	What this is: Registration trusts whatever fields the client sends, including the account role. Problem: A user can add role=admin to the signup request and become an administrator. If we skip it: Anyone can create an admin account at will.
Description	The registration create hook accepts a client-supplied role, so sending role=admin creates an admin account
Business Impact	Privilege escalation to administrator from an anonymous signup.
Root Cause	The create path does not strip/override the client-controllable role attribute.
Evidence (source trace)	`1. server.ts:550 (user creation accepts client-provided fields including role)` `resource.create.send.before((req: Request, res: Response, context: { instance: { id: any }, continue: any }) => { // vuln-code-snippet vuln-line registerAdminChallenge`
Recommendation	Whitelist the registration fields and hard-set role to 'customer' on the server. Files to change: server.ts, models/user.ts
Fix example	`context.instance.role = 'customer'`
Production risk of the fix	Any legitimate flow that set role at creation must move to an admin-only path.
Regression test	Add a test registering with role=admin and assert the created user is a customer.
Classification	CWE-915, CWE-269 | A01:2021 Broken Access Control, API3:2023 | CVSS 9.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-01 SQL injection in login allows authentication bypass and full Users table disclosure [HIGH]

Location	routes/login.ts:34

In plain words	What this is: The login screen builds its database query by pasting your email straight into the SQL text. Problem: Because the email is not treated as data, typing SQL into it rewrites the query, so anyone can log in as the admin without a password. If we skip it: Anyone on the internet can log in as any user, including admin, and read every account's email and password hash.
Description	An attacker logs in as admin or any user with ' OR 1=1-- and can extract every row of the Users table (emails, MD5 password hashes)
Business Impact	Complete authentication bypass and disclosure of all customer credentials; full account takeover of the admin.
Root Cause	User input is interpolated into a raw SQL string instead of being passed as a bound parameter.
Evidence (source trace)	`1. routes/login.ts:34 (email concatenated straight into SQL; no binding)` `models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })`
Recommendation	Replace the template-literal query with a parameterised Sequelize query using replacements, and compare the password in the WHERE clause only via a bound value. Files to change: routes/login.ts
Fix example	`models.sequelize.query('SELECT * FROM Users WHERE email = :email AND password = :pw AND deletedAt IS NULL', { replacements: { email: req.body.email, pw: security.hash(req.body.password) }, model: UserModel, plain: true })`
Production risk of the fix	Parameterisation is behaviour-preserving for legitimate logins; only inputs that previously injected SQL stop working.
Regression test	Add a test that POSTs email "' OR 1=1--" and asserts the response is 401, not a token.
Classification	CWE-89 | A03:2021 Injection, API8:2023 | CVSS 8.8 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-02 SQL injection in product search enables UNION-based database exfiltration [HIGH]

Location	routes/search.ts:23
In plain words	What this is: The product search pastes your search text directly into the database query. Problem: A crafted search term can bolt a second query on with UNION and read any table, and can reveal hidden/deleted products. If we skip it: Anyone can dump the database, including user emails and password hashes, through the search box.

Description	A UNION SELECT payload in q returns arbitrary columns/tables (user credentials, schema), and reveals soft-deleted products
Business Impact	Full read access to the database (credentials, schema, hidden products) from an unauthenticated endpoint.
Root Cause	The q parameter is interpolated into a raw SQL LIKE clause with no binding.
Evidence (source trace)	`1. routes/search.ts:23 (criteria from req.query.q concatenated into SQL) models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)`
Recommendation	Use a parameterised query with a bound LIKE value and escaped wildcards. Files to change: routes/search.ts
Fix example	`models.sequelize.query('SELECT * FROM Products WHERE (name LIKE :q OR description LIKE :q) AND deletedAt IS NULL ORDER BY name', { replacements: { q: `%${criteria}%` } })`
Production risk of the fix	Behaviour-preserving for ordinary searches; only injection payloads change behaviour.
Regression test	Add a test searching q=")) UNION SELECT ... -- and assert no extra columns/tables are returned.
Classification	CWE-89 | A03:2021 Injection | CVSS 8.7 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-03 NoSQL injection via MongoDB $where in product reviews and order tracking [HIGH]

Location	routes/showProductReviews.ts:36
In plain words	What this is: Reviews and order lookups build a MongoDB query by gluing your id into a snippet of JavaScript the database runs. Problem: Because the id becomes code, an attacker can return other people's orders or hang the database with an infinite loop. If we skip it: Attackers read arbitrary orders and can take the shop offline with a crafted id.
Description	A JavaScript payload in id runs inside the Mongo $where clause, allowing data exfiltration across orders and a sleep() denial of service
Business Impact	Cross-customer data exfiltration and a denial-of-service lever, both unauthenticated.
Root Cause	User input is concatenated into a MongoDB $where JavaScript expression instead of a value comparison.

Evidence (source trace)	`1. routes/showProductReviews.ts:36 (id concatenated into a $where JavaScript expression)`
	`db.reviewsCollection.find({ $where: 'this.product == ' + id }).then((reviews: Review[]) => {`
	`2. routes/trackOrder.ts:18 (same $where injection on order id)`
	`db.ordersCollection.find({ $where: `this.orderId === '${id}'` }).then((order: any) => {`

Recommendation	Replace $where with a direct field match, e.g. find({ product: Number(id) }) / find({ orderId: id }). Files to change: routes/showProductReviews.ts, routes/trackOrder.ts
Fix example	`db.reviewsCollection.find({ product: Number(id) })`
Production risk of the fix	Equivalent results for legitimate ids; rejects non-numeric ids that previously 'worked' via injection.
Regression test	Add a test passing id='0;while(true){}' and assert it is rejected and does not hang.
Classification	CWE-943 | A03:2021 Injection | CVSS 8.8 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:L/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-04 NoSQL injection and missing authorization let any user overwrite all product reviews [HIGH]

Location	routes/updateProductReviews.ts:18, routes/updateProductReviews.ts:20
In plain words	What this is: Editing a review trusts the id you send and updates every matching row. Problem: Sending a query operator instead of an id matches all reviews, and the multi flag overwrites them all at once. If we skip it: Any user can replace the text of every product review in the shop.
Description	A Mongo operator in id (e.g. {$ne:-1}) matches every review and {multi:true} rewrites all of them in one request
Business Impact	Integrity loss across all product reviews; one request defaces the entire catalogue's feedback.
Root Cause	Unvalidated req.body.id used as a Mongo selector together with multi:true and no ownership check.

Evidence (source trace)	`1. routes/updateProductReviews.ts:18 (req.body.id passed as selector with no type/operator check)`
	`{ _id: req.body.id },`
	`2. routes/updateProductReviews.ts:20 (update applies to every matched document)`
	`{ multi: true }`

Recommendation	Coerce id to a single ObjectId, remove multi:true, and verify the review's author matches the caller. Files to change: routes/updateProductReviews.ts

Fix example	`db.reviewsCollection.update({ _id: String(req.body.id) }, { $set: { message } })`
Production risk of the fix	Legitimate single-review edits are unaffected; bulk/operator updates stop working.
Regression test	Add a test sending id={$ne:-1} and assert only the targeted review (or none) is updated.
Classification	CWE-943, CWE-639 | A01:2021 Broken Access Control, A03:2021 Injection | CVSS 7.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-09 XML external entity (XXE) processing enables file disclosure and denial of service [HIGH]

Location	lib/xml.ts:35, lib/xml.ts:38
In plain words	What this is: The server parses uploaded XML with external entity resolution switched on. Problem: A crafted XML file can read server files (like /etc/passwd) or exhaust memory/CPU. If we skip it: Attackers read local files from the server or knock it offline with an entity-expansion bomb.
Description	The parser is configured with NOENT|DTDLOAD and filesystem providers, so an XXE payload reads local files or expands a billion-laughs bomb
Business Impact	Local file disclosure and denial of service from an authenticated upload.
Root Cause	The XML parser is explicitly configured to resolve external entities and load DTDs, with filesystem access.

Evidence (source trace)	`1. lib/xml.ts:35 (entity substitution and DTD loading enabled)`
	`const option = libxml2.ParseOption.XML_PARSE_NOENT | libxml2.ParseOption.XML_PARSE_DTDLOAD | libxml2.ParseOption.XML_PARSE_NOBLANKS | libxml2.ParseOption.XML_PARSE_NOCDATA`
	`2. lib/xml.ts:38 (parses attacker XML with those options)`
	`const xmlDoc = vm.runInContext('libxml2.XmlDocument.fromString(data, { option })', sandbox, { timeout: timeoutMs })`
	`3. routes/fileUpload.ts:80 (entry point: parses uploaded XML)`
	`const xmlString = await parseXmlString(data)`

Recommendation	Parse with entity expansion and DTD loading disabled and without filesystem input providers. Files to change: lib/xml.ts
Fix example	`const option = libxml2.ParseOption.XML_PARSE_NOBLANKS // no NOENT, no DTDLOAD`
Production risk of the fix	XML that legitimately relied on external entities (rare) would need to be inlined.

Regression test	Add a test uploading an XXE payload referencing a local file and assert the file content is not returned.
Classification	CWE-611 | A05:2021 Security Misconfiguration | CVSS 7.2 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:H/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-10 Denial of service via unbounded YAML deserialization of uploaded files [HIGH]

Location	routes/fileUpload.ts:114
In plain words	What this is: Uploaded YAML files are fully parsed with no size or depth limit. Problem: A tiny file using repeated YAML anchors expands to gigabytes and exhausts the server. If we skip it: One small upload can take the shop offline.
Description	A YAML bomb (recursive anchors) is loaded inside a 2s VM, exhausting memory/CPU and stalling the worker
Business Impact	Single-request denial of service against the application.
Root Cause	User-supplied YAML is deserialized with no bound on anchor/alias expansion.
Evidence (source trace)	`1. routes/fileUpload.ts:114 (loads attacker YAML; anchor expansion blows up before the timeout helps)` `const yamlString = vm.runInContext('JSON.stringify(yaml.load(data))', sandbox, { timeout: 2000 })`
Recommendation	Enforce a strict byte-size limit and use a YAML loader configuration that disallows/limits aliases before parsing. Files to change: routes/fileUpload.ts
Fix example	`if (data.length > 100_000) throw new Error('file too large');` `yaml.load(data, { ... safe limits ... })`
Production risk of the fix	Legitimate large or anchor-heavy YAML uploads would be rejected.
Regression test	Add a test uploading a billion-laughs YAML and assert the request is rejected quickly without memory spike.
Classification	CWE-400, CWE-776 | A05:2021 Security Misconfiguration | CVSS 7.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-11 Server-side code execution via eval of B2B order data [HIGH]

Location	routes/b2bOrder.ts:23

In plain words	What this is: B2B orders are processed by evaluating the submitted data as JavaScript. Problem: Evaluating input as code lets an attacker run their own code or hang the server. If we skip it: Attackers can execute code on the server or take it offline.
Description	orderLinesData is passed to safeEval inside a VM; a crafted payload runs attacker JavaScript and can hang the worker (DoS) or break out
Business Impact	Potential remote code execution and reliable denial of service on the order pipeline.
Root Cause	Attacker-controlled order data is evaluated via safeEval/vm instead of being parsed as data.
Evidence (source trace)	`1. routes/b2bOrder.ts:23 (attacker-controlled orderLinesData evaluated as code)` `vm.runInContext('safeEval(orderLinesData)', sandbox, { timeout: 2000 })`
Recommendation	Replace the eval with JSON.parse plus schema validation (e.g. zod) of the order lines. Files to change: routes/b2bOrder.ts
Fix example	`const orderLines = OrderLinesSchema.parse(JSON.parse(req.body.orderLinesData))`
Production risk of the fix	Clients that sent non-JSON expressions must switch to plain JSON payloads.
Regression test	Add a test POSTing orderLinesData='while(true){}' and assert it is rejected without hanging.
Classification	CWE-94, CWE-915 | A03:2021 Injection | CVSS 8.7 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-12 Server-side template injection and unsafe CSP via the username field [HIGH]

Location	routes/userProfile.ts:65, routes/userProfile.ts:91
In plain words	What this is: The profile page runs the username through a template engine that can execute code, and builds a weak content-security policy from user data. Problem: A specially formatted username executes on the server, and the page's CSP allows unsafe-eval and reflects user input. If we skip it: Attackers run code on the server and bypass XSS protections in the browser.
Description	A username matching #{...} is eval'd server-side, and the profile page sets a CSP that reflects the username and allows unsafe-eval
Business Impact	Server-side code execution and a browser-side script-execution bypass from a stored profile field.

Root Cause	The username is evaluated as a Pug/JS template and reflected into a permissive CSP header.

Evidence (source trace)	`1. routes/userProfile.ts:65 (username expression eval'd server-side)`
	`username = eval(code) // eslint-disable-line no-eval`
	`2. routes/userProfile.ts:91 (user data reflected into CSP; unsafe-eval allowed)`
	`const CSP = `img-src 'self' ${user?.profileImage}; script-src 'self' 'unsafe-eval'``

Recommendation	Remove the eval, render the username as escaped text, and send a fixed strict CSP without 'unsafe-eval' or reflected values. Files to change: routes/userProfile.ts
Fix example	`res.set('Content-Security-Policy', "default-src 'self'; script-src 'self'")`
Production risk of the fix	Any feature that depended on expressions in usernames or inline eval must be reworked.
Regression test	Add a test setting username to #{7*7} and assert the response contains the literal, not 49, and the CSP has no unsafe-eval.
Classification	CWE-1336, CWE-94 | A03:2021 Injection | CVSS 8.7 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-19 Product records can be modified because the PUT route is left unprotected [HIGH]

Location	server.ts:402
In plain words	What this is: The rule that should require permission to edit a product is commented out. Problem: So any logged-in user can change product details, including injecting HTML into descriptions. If we skip it: Customers can tamper with the catalogue and plant stored XSS.
Description	The authorization line for PUT /api/Products is commented out, so any user can tamper with product data (name, price, description with XSS)
Business Impact	Catalogue integrity loss and a stored-XSS delivery path via product descriptions.
Root Cause	The isAuthorized() guard on PUT /api/Products/:id is commented out.
Evidence (source trace)	`1. server.ts:402 (authorization for product update is disabled)` `// app.put('/api/Products/:id', security.isAuthorized()) // vuln-code-snippet vuln-line changeProductChallenge`
Recommendation	Restore the authorization middleware (and role check) on the product update route. Files to change: server.ts
Fix example	`app.put('/api/Products/:id', security.isAuthorized())`

Production risk of the fix	Clients that updated products without auth (none legitimate) will now get 401.
Regression test	Add a test that a customer PUT to /api/Products/1 returns 401.
Classification	CWE-862 | A01:2021 Broken Access Control, API5:2023 | CVSS 7.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-23 Passkey registration overwrites existing credentials and session is issued regardless of verification [HIGH]

Location	routes/passkey.ts:142, routes/passkey.ts:222
In plain words	What this is: The passkey (WebAuthn) flow lets one credential overwrite another and hands out a session even when the security check did not pass. Problem: That allows hijacking another account's passkey and logging in without a valid signature. If we skip it: Attackers take over accounts through the passkey flow.
Description	Registration overwrites any existing credential by id, and the login path issues a session even when verification did not succeed
Business Impact	Account takeover via passkey hijack and signature-check bypass.
Root Cause	Credential records are keyed without owner binding, and session issuance is not gated on verification success.

Evidence (source trace)	`1. routes/passkey.ts:142 (existing credential overwritten without owner binding)`
	`existing.publicKey = publicKey // vuln-code-snippet vuln-line passkeyCredentialOverwriteChallenge`
	`2. routes/passkey.ts:222 (session issued outside the verified branch)`
	`await issuePasskeySession(user, res) // vuln-code-snippet vuln-line passkeySignatureChallenge`

Recommendation	Reject registration when the credentialID belongs to another user, and move issuePasskeySession inside the verification.verified branch. Files to change: routes/passkey.ts
Fix example	`if (!verification.verified) return res.sendStatus(401); await issuePasskeySession(user, res)`
Production risk of the fix	Logins that previously succeeded without a valid signature will now fail (as intended).
Regression test	Add a test where verification.verified is false and assert no session cookie is issued.
Classification	CWE-287, CWE-639 | A07:2021 Identification and Authentication Failures | CVSS 8.6 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-24 Password hash and other sensitive fields leak via the currentUser fields parameter [HIGH]

Location	routes/currentUser.ts:22, routes/currentUser.ts:31
In plain words	What this is: The 'who am I' API lets the client pick which fields to return, with no safe-list. Problem: Asking for the password field returns the stored password hash. If we skip it: Users can read their (and via other flaws, others') password hashes.
Description	The endpoint copies any requested field from the user object, so fields=password returns the stored password hash
Business Impact	Disclosure of password hashes and TOTP secrets to any authenticated caller.
Root Cause	The response is built from a client-controlled field list with no allowlist.

Evidence (source trace)	`1. routes/currentUser.ts:22  (client chooses which fields to return)`
	`  const fieldsParam = req.query?.fields as string | undefined`
	`2. routes/currentUser.ts:31  (no allowlist; copies password/totpSecret if asked)`
	`  baseUser[field] = user?.data[field as keyof typeof user.data]`

Recommendation	Replace the dynamic field projection with a fixed allowlist of non-sensitive fields. Files to change: routes/currentUser.ts
Fix example	`const safe = { id: user.data.id, email: user.data.email, role: user.data.role }`
Production risk of the fix	Clients requesting non-allowlisted fields stop receiving them.
Regression test	Add a test requesting fields=password and assert the hash is not in the response.
Classification	CWE-213, CWE-639 | A01:2021 Broken Access Control, API3:2023 | CVSS 7.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-26 Password can be changed without knowing the current password [HIGH]

Location	routes/changePassword.ts:39
In plain words	What this is: The change-password feature only checks your current password if you bother to send one. Problem: Leaving it out skips the check, so a forged request can set a new password. If we skip it: An attacker who can trigger a request in the victim's session takes over the account.
Description	The current-password check is skipped when current is empty, so a request without it still changes the password
Business Impact	Account takeover via CSRF or any flow that omits the current password.

Root Cause	The current-password comparison is guarded by a truthiness check, making it optional.
Evidence (source trace)	`1. routes/changePassword.ts:39 (check only runs when currentPassword is truthy; omit it to skip)` `if (currentPassword && security.hash(currentPassword) !== loggedInUser.data.password) {`
Recommendation	Require current to be present and correct before changing the password; reject if missing. Files to change: routes/changePassword.ts
Fix example	`if (!currentPassword || security.hash(currentPassword) !== loggedInUser.data.password) return res.status(401)...`
Production risk of the fix	Clients that changed passwords without supplying the current one must be updated.
Regression test	Add a test changing the password with no current value and assert 401.
Classification	CWE-620, CWE-352 | A07:2021 Identification and Authentication Failures | CVSS 7.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-07 Passwords stored as unsalted MD5 hashes [MEDIUM]

Location	lib/insecurity.ts:41
In plain words	What this is: Passwords are scrambled with MD5, an old and fast algorithm with no salt. Problem: MD5 hashes can be reversed almost instantly, so any leak of the table exposes real passwords. If we skip it: A single database leak hands attackers every user's actual password.
Description	MD5 hashes are trivially cracked / rainbow-tabled, so a leaked Users table yields plaintext passwords
Business Impact	Any disclosure of the password column leads to mass credential compromise and password-reuse attacks.
Root Cause	Password hashing uses unsalted MD5 instead of a modern password KDF.

Evidence (source trace)	`1. lib/insecurity.ts:41 (MD5, no salt)`
	`export const hash = (data: string) => crypto.createHash('md5').update(data).digest('hex')`
	`2. models/user.ts:73 (password set via the MD5 hash helper)`
	`this.setDataValue('password', security.hash(clearTextPassword)) // vuln-code-snippet vuln-line weakPasswordChallenge`

Recommendation	Replace MD5 with bcrypt or argon2id and rehash on successful login. Files to change: lib/insecurity.ts, models/user.ts

Fix example	`import bcrypt from 'bcryptjs'; const hash = await bcrypt.hash(clearTextPassword, 12)`
Production risk of the fix	Needs a migration path; verify against the old MD5 hash once, then store the new hash.
Regression test	Add a test asserting a newly set password does not equal its MD5 digest.
Classification	CWE-916, CWE-327 | A02:2021 Cryptographic Failures | CVSS 6.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-08 Open redirect: allowlist uses substring match on the full URL [MEDIUM]

Location	lib/insecurity.ts:136
In plain words	What this is: The redirect feature checks the destination by looking for a trusted URL anywhere inside it. Problem: An attacker puts a trusted URL in a query string so their malicious link passes the check. If we skip it: The shop can be used to bounce victims to phishing/malware sites under its own name.
Description	Because the check is url.includes(allowedUrl), http://evil.tld/?x=https://github.com/... passes and redirects victims off-site
Business Impact	Phishing and token-leak vector that abuses the shop's trusted domain.
Root Cause	The allowlist is enforced with a substring check instead of an exact-origin comparison.
Evidence (source trace)	`1. lib/insecurity.ts:136  (substring match, not origin match)` `allowed = allowed || url.includes(allowedUrl) // vuln-code-snippet vuln-line redirectChallenge`
Recommendation	Parse the candidate URL and require new URL(url).origin to exactly equal an allowlisted origin. Files to change: lib/insecurity.ts
Fix example	`const o = new URL(url).origin; allowed = allowlist.some(a => new URL(a).origin === o)`
Production risk of the fix	Links that previously passed only by containing an allowlisted substring stop redirecting.
Regression test	Add a test that http://evil.tld/?x=<allowlisted> is rejected.
Classification	CWE-601 | A01:2021 Broken Access Control | CVSS 5.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:N/VI:L/VA:N/SC:L/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-13 Server-side request forgery via profile image URL fetch [MEDIUM]

Location	routes/profileImageUrlUpload.ts:24
In plain words	What this is: Setting a profile image by URL makes the server download whatever address you give it. Problem: With no restriction, an attacker points it at internal systems or cloud metadata to reach things the server can but they cannot. If we skip it: Attackers use the server as a proxy into the internal network / cloud metadata.
Description	The server fetches an attacker-supplied URL with no host allowlist, reaching internal services/metadata endpoints
Business Impact	Pivot into internal services and potential cloud-credential theft via metadata endpoints.
Root Cause	A user-supplied URL is fetched with no scheme/host validation or SSRF protections.
Evidence (source trace)	`1. routes/profileImageUrlUpload.ts:24  (user-controlled url fetched with no allowlist/scheme check)` `const response = await fetch(url)`
Recommendation	Validate the URL scheme and resolved IP (reject private/link-local ranges), disable redirects, and verify the response content-type is an image. Files to change: routes/profileImageUrlUpload.ts
Fix example	`if (!/^https:\/\//.test(url) || isPrivateHost(new URL(url).hostname)) throw new Error('blocked')`
Production risk of the fix	Image URLs on non-public hosts or over http stop working.
Regression test	Add a test setting imageUrl to http://169.254.169.254/ and assert the fetch is blocked.
Classification	CWE-918 | A10:2021 SSRF | CVSS 5.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:N/VA:N/SC:L/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-14 DOM XSS via DomSanitizer.bypassSecurityTrustHtml on untrusted data [MEDIUM]

Location	frontend/src/app/search-result/search-result.component.ts:146, frontend/src/app/search-result/search-result.component.ts:113
In plain words	What this is: Several screens explicitly tell Angular to trust attacker-influenced text as raw HTML. Problem: That disables the built-in escaping, so injected markup runs as script in the victim's browser. If we skip it: Attackers run script in other users' sessions (account/session theft).

Description	Several components wrap untrusted values in bypassSecurityTrustHtml, disabling Angular's escaping and executing injected HTML/JS
Business Impact	Stored and reflected XSS leading to session theft and account takeover of other users, including admins.
Root Cause	DomSanitizer.bypassSecurityTrustHtml is used on untrusted values, defeating Angular's output encoding.

Evidence (source trace)	`1. frontend/src/app/search-result/search-result.component.ts:146 (reflected search term trusted as HTML)`
	`this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam) // vuln-code-snippet vuln-line localXssChallenge xssBonusChallenge`
	`2. frontend/src/app/search-result/search-result.component.ts:113 (product description trusted as HTML)`
	`tableData[i].description = this.sanitizer.bypassSecurityTrustHtml(tableData[i].description) // vuln-code-snippet vuln-line restfulXssChallenge`
	`3. frontend/src/app/track-result/track-result.component.ts:49 (order id trusted as HTML (reflected XSS))`
	`this.results.orderNo = this.sanitizer.bypassSecurityTrustHtml(`<code>${results.data[0].orderId}</code>`)`
	`4. frontend/src/app/last-login-ip/last-login-ip.component.ts:40 (last-login IP (from a header) trusted as HTML (HTTP-header XSS))`
	`this.lastLoginIp = this.sanitizer.bypassSecurityTrustHtml(`${payload.data.lastLoginIp}`)`

Recommendation	Remove the bypassSecurityTrust* calls and render values via normal interpolation (which escapes), or sanitize explicitly when HTML is required. Files to change: frontend/src/app/search-result/search-result.component.ts, frontend/src/app/track-result/track-result.component.ts, frontend/src/app/last-login-ip/last-login-ip.component.ts
Fix example	`// template: <span>{{ searchValue }}</span> // no bypassSecurityTrustHtml`
Production risk of the fix	Any intentional rich-HTML rendering in these fields would need a vetted sanitizer.
Regression test	Add a test rendering a product description of <img src=x onerror=alert(1)> and assert no script executes.
Classification	CWE-79 | A03:2021 Injection | CVSS 5.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:L/VI:L/VA:N/SC:L/SI:L/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-15 Directory listing and public download of the FTP folder expose forgotten backups [MEDIUM]

Location	server.ts:294

In plain words	What this is: A server folder is browsable and downloadable by anyone, and it still holds old backup files. Problem: Those backups leak source, dependency lists and historical coupon codes. If we skip it: Anyone can enumerate and download sensitive server files.
Description	/ftp is directory-listed and serves files; forgotten backups (package.json.bak, coupons_2013.md.bak) are downloadable, leaking code and coupon logic
Business Impact	Disclosure of source/dependency metadata and coupon logic that aids further attacks.
Root Cause	serveIndex exposes the ftp/ directory and backups were left inside a web-served path.

Evidence (source trace)	`1. server.ts:294 (anonymous directory listing of ftp/)`
	`app.use('/ftp', serveIndexMiddleware, serveIndex('ftp', { icons: true })) // vuln-code-snippet vuln-line directoryListingChallenge`
	`2. ftp/package.json.bak:2 (forgotten developer backup, web-downloadable)`
	`"name": "juice-shop",`
	`3. ftp/coupons_2013.md.bak:1 (forgotten sales backup (coupon codes), web-downloadable)`
	`n<MibgC7sn`

Recommendation	Remove the serveIndex listing, delete backup files from ftp/, and serve only an explicit allowlist of files. Files to change: server.ts
Fix example	`// remove app.use('/ftp', serveIndex(...)) and purge *.bak from ftp/`
Production risk of the fix	Any client relying on browsing /ftp must switch to explicit download links.
Regression test	Add a test that GET /ftp returns no listing and GET /ftp/package.json.bak returns 404.
Classification	CWE-548, CWE-538 | A05:2021 Security Misconfiguration | CVSS 6.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-16 Server access logs are exposed through a browsable /support/logs directory [MEDIUM]

Location	server.ts:306
In plain words	What this is: The server's access-log folder can be browsed and downloaded by anyone. Problem: Logs reveal who did what, their IPs, and any secrets that appeared in URLs. If we skip it: Attackers harvest user activity and leaked credentials from logs.
Description	/support/logs is directory-listed and serves log files, leaking request history, IPs and tokens-in-URLs
Business Impact	Disclosure of user activity, IP addresses and any URL-borne secrets.

Root Cause	serveIndex exposes the logs/ directory to anonymous users.
Evidence (source trace)	`1. server.ts:306 (anonymous directory listing of logs/)` `app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view: 'details' })) // vuln-code-snippet vuln-line accessLogDisclosureChallenge`
Recommendation	Remove the /support/logs listing and route and keep logs outside any web-served path. Files to change: server.ts
Fix example	`// remove app.use('/support/logs', serveIndex(...))`
Production risk of the fix	Support tooling that read logs via the web must use a proper admin channel.
Regression test	Add a test that GET /support/logs returns 404/empty, not a file listing.
Classification	CWE-532, CWE-548 | A09:2021 Security Logging and Monitoring Failures | CVSS 6.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-17 Prometheus metrics endpoint is exposed without authentication [MEDIUM]

Location	server.ts:771
In plain words	What this is: The internal metrics page is open to the public. Problem: It leaks operational detail attackers can use to profile the system and users. If we skip it: Attackers read internal metrics and user/activity counts.
Description	/metrics is served to anyone, leaking internal counters, user/challenge counts and infrastructure signals
Business Impact	Operational and user-activity information disclosure that aids targeting.
Root Cause	The /metrics route is registered with no authentication or network restriction.
Evidence (source trace)	`1. server.ts:771 (no auth guard on /metrics)` `app.get('/metrics', utils.asyncHandler(metrics.serveMetrics())) // vuln-code-snippet vuln-line exposedMetricsChallenge`
Recommendation	Protect /metrics behind authentication or bind it to an internal interface/allowlist. Files to change: server.ts
Fix example	`app.get('/metrics', security.isAuthorized(), utils.asyncHandler(metrics.serveMetrics()))`
Production risk of the fix	The scraping agent must be given credentials or network access.
Regression test	Add a test that GET /metrics without auth returns 401.

Classification	CWE-200 | A01:2021 Broken Access Control | CVSS 6.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-20 Sensitive routes are gated only by client-side guards (admin, scoreboard, web3, token sale) [MEDIUM]

Location	frontend/src/app/app.routing.ts:80, frontend/src/app/app.routing.ts:187
In plain words	What this is: Admin and other non-public pages are only hidden/guarded in the browser code. Problem: Anyone can read the routes and open them directly; client-side guards do not protect server data. If we skip it: Attackers reach admin and hidden areas by typing the URL.
Description	The admin section, score board, web3 sandbox and token-sale routes are defined client-side (some behind a client-only guard or an obfuscated matcher), reachable by navigating directly
Business Impact	Exposure of administrative and hidden functionality that should require server-enforced authorization.
Root Cause	Access control for these areas lives only in the Angular route table, not on the server.

Evidence (source trace)	`1. frontend/src/app/app.routing.ts:80 (admin route only guarded by a client-side AdminGuard)`
	`path: 'administration', // vuln-code-snippet vuln-line adminSectionChallenge`
	`2. frontend/src/app/app.routing.ts:187 (hidden score board route)`
	`path: 'score-board', // vuln-code-snippet vuln-line scoreBoardChallenge`
	`3. frontend/src/app/app.routing.ts:245 (undocumented web3 sandbox route)`
	`path: 'web3-sandbox', // vuln-code-snippet vuln-line web3SandboxChallenge`
	`4. frontend/src/app/app.routing.ts:267 (token-sale route hidden behind an obfuscated URL matcher)`
	`matcher: tokenMatcher, // vuln-code-snippet vuln-line tokenSaleChallenge`

Recommendation	Keep the client guards but enforce role/authorization server-side on every endpoint these pages call. Files to change: server.ts
Fix example	`// server: app.use('/rest/admin/*', security.isAuthorized(), security.isAdmin())`
Production risk of the fix	Any client that reached these endpoints without proper auth will now be rejected.
Regression test	Add a server test that admin endpoints require an admin token.

Classification	CWE-602, CWE-656 | A01:2021 Broken Access Control | CVSS 6.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-21 Chatbot coupon tool can be driven by prompt injection to mint discounts [MEDIUM]

Location	routes/chat.ts:180, routes/chat.ts:185
In plain words	What this is: The support chatbot can create discount coupons, and the only limit is a sentence in its instructions. Problem: A user can talk the bot into issuing bigger discounts than allowed because the cap is not enforced in code. If we skip it: Users obtain arbitrary discounts, causing revenue loss.
Description	The coupon tool trusts the model's discount argument and the system prompt only states the cap in text, so an injected message makes the bot emit larger discounts
Business Impact	Financial loss through attacker-controlled discount coupons.
Root Cause	The discount value from the LLM tool call is used without a server-side bound.
Evidence (source trace)	`1. routes/chat.ts:180 (max is only described in text, not enforced) discount: z.number().describe('The discount percentage for the coupon (maximum 10)') // vuln-code-snippet vuln-line chatbotPromptInjectionChallenge chatbotGreedyInjectionChallenge` `2. routes/chat.ts:185 (discount from the model used directly) const couponCode = security.generateCoupon(discount) // vuln-code-snippet vuln-line chatbotPromptInjectionChallenge`
Recommendation	Enforce the maximum discount in code before calling generateCoupon, independent of the model output. Files to change: routes/chat.ts
Fix example	`const safe = Math.min(Math.max(discount, 0), 10); const couponCode = security.generateCoupon(safe)`
Production risk of the fix	The bot can no longer grant above-policy discounts even if a human expected it to.
Regression test	Add a test where the model returns discount=80 and assert the generated coupon encodes at most 10.
Classification	CWE-284, CWE-20 | LLM01:2025 Prompt Injection | CVSS 5.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-22 Path traversal via poison null byte in the FTP file server [MEDIUM]

Location	routes/fileServer.ts:32

In plain words	What this is: The file download only allows .md/.pdf, but a special null-byte trick fools that check. Problem: Attackers append %00 to grab blocked files like the developer backup. If we skip it: The extension allowlist is bypassable and sensitive files leak.
Description	A %00 poison null byte (e.g. package.json.bak%2500.md) bypasses the extension allowlist and downloads otherwise-blocked files
Business Impact	Disclosure of non-whitelisted server files (e.g. the forgotten developer backup).
Root Cause	The extension allowlist is applied to a string that can be truncated by an encoded null byte.
Evidence (source trace)	`1. routes/fileServer.ts:32  (file served after a null-byte-truncatable extension check)` `res.sendFile(path.resolve('ftp/', file))`
Recommendation	Reject requests whose decoded filename contains a null byte, and check the extension on the decoded, normalised name. Files to change: routes/fileServer.ts
Fix example	`if (file.includes('\0') || file.includes('%00')) return res.sendStatus(400)`
Production risk of the fix	None for legitimate .md/.pdf downloads.
Regression test	Add a test requesting /ftp/package.json.bak%2500.md and assert 400/404.
Classification	CWE-22, CWE-158 | A01:2021 Broken Access Control | CVSS 6.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-25 Password-reset rate limit is bypassable via the X-Forwarded-For header [MEDIUM]

Location	server.ts:378
In plain words	What this is: The password-reset throttle counts attempts per X-Forwarded-For header. Problem: Because the attacker sets that header, they can send unlimited attempts by changing it each time. If we skip it: Attackers brute-force security answers and reset others' passwords.
Description	The limiter keys on X-Forwarded-For, which the client controls, so rotating the header evades the limit and enables brute force of security answers
Business Impact	Brute-force of the account-recovery flow leading to account takeover.
Root Cause	The rate-limit key is derived from a client-controlled header instead of the trusted peer IP.

Evidence (source trace)	`1. server.ts:378 (rate-limit key is attacker-controlled) keyGenerator ({ headers, ip }: { headers: any, ip: any }) { return headers['X-Forwarded-For'] ?? ip } // vuln-code-snippet vuln-line resetPasswordMortyChallenge`
Recommendation	Configure Express trust proxy and key the limiter on req.ip, ignoring raw X-Forwarded-For. Files to change: server.ts
Fix example	`keyGenerator: (req) => req.ip`
Production risk of the fix	Behind a real proxy, set trust proxy correctly so req.ip is the true client.
Regression test	Add a test sending many resets with rotating X-Forwarded-For and assert the limiter still blocks.
Classification	CWE-307, CWE-290 | A07:2021 Identification and Authentication Failures | CVSS 6.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-27 Known-vulnerable and end-of-life dependencies are pinned (sanitize-html 1.4.2, jsonwebtoken 0.4.0, express-jwt 0.1.3) [MEDIUM]

Location	package.json:136, package.json:123
In plain words	What this is: Several libraries are pinned to very old versions with publicly known weaknesses. Problem: The HTML sanitiser can be bypassed (XSS) and the old JWT libraries mishandle algorithms. If we skip it: Attackers exploit documented bypasses in these components.
Description	sanitize-html 1.4.2 has known XSS-filter bypasses, and jsonwebtoken 0.4.0 / express-jwt 0.1.3 are end-of-life with algorithm-handling weaknesses
Business Impact	XSS-filter bypass and weakened token handling via outdated dependencies.
Root Cause	Dependencies are pinned to end-of-life versions with known advisories.

Evidence (source trace)	`1. package.json:136 (known XSS sanitiser bypasses in this version) "sanitize-html": "1.4.2",`
	`2. package.json:123 (end-of-life; algorithm-confusion weaknesses) "jsonwebtoken": "0.4.0",`
	`3. package.json:109 (end-of-life JWT middleware) "express-jwt": "0.1.3",`

Recommendation	Upgrade sanitize-html, jsonwebtoken and express-jwt to maintained versions and re-test the sanitisation and token paths. Files to change: package.json
Fix example	`// package.json: "sanitize-html": "^2.13.0", "jsonwebtoken": "^9.0.2"`

Production risk of the fix	Major-version upgrades may change sanitiser output and JWT API calls; test accordingly.
Regression test	Add a CI check (npm audit / osv-scanner) that fails on known high-severity advisories in these packages.
Classification	CWE-1035, CWE-937 | A06:2021 Vulnerable and Outdated Components | CVSS 6.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N) | Confidence: High | Votes: 3 confirmed / 0 verify / 0 reject

F-28 Typosquatted frontend dependency present (ngy-cookie) [MEDIUM]

Location	frontend/package.json:59
In plain words	What this is: A dependency name is a near-copy of a real package (ngy-cookie vs ngx-cookie). Problem: Typosquatted packages are a common way to slip malicious code into an app. If we skip it: Malicious code in the lookalike package runs in the build and in users' browsers.
Description	frontend/package.json depends on ngy-cookie, a typosquat of the legitimate ngx-cookie package, which can ship malicious code into the build
Business Impact	Supply-chain compromise risk affecting the built frontend delivered to all users.
Root Cause	A lookalike (typosquatted) package is declared instead of the legitimate one.
Evidence (source trace)	`1. frontend/package.json:59  (typosquat of ngx-cookie)` `"ngy-cookie": "^6.0.0",`
Recommendation	Remove ngy-cookie, install the legitimate ngx-cookie (or an alternative), and audit the lockfile for other lookalikes. Files to change: frontend/package.json
Fix example	`// frontend/package.json: replace "ngy-cookie" with the vetted "ngx-cookie"`
Production risk of the fix	Import paths referencing the old package name must be updated.
Regression test	Add a CI allowlist/denylist check that fails when ngy-cookie is present.
Classification	CWE-1357, CWE-829 | A06:2021 Vulnerable and Outdated Components | CVSS 6.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N) | Confidence: Medium | Votes: 2 confirmed / 1 verify / 0 reject (dissent: One verifier wanted registry confirmation that ngy-cookie ships malicious code rather than being a benign lookalike.)

F-29 Stored XSS via video subtitle template injection [MEDIUM]

Location	routes/videoHandler.ts:71
In plain words	What this is: The video page drops subtitle text straight into a script tag. Problem: Unescaped subtitle content can include script that runs in the viewer's browser. If we skip it: Attackers run script in viewers' sessions via the video page.
Description	Subtitle text is concatenated into a compiled Pug template and emitted inside a <script> block without escaping, executing attacker markup
Business Impact	Stored XSS affecting anyone who views the video page.
Root Cause	Subtitle text is concatenated into the page/script without output encoding.
Evidence (source trace)	`1. routes/videoHandler.ts:71  (subs concatenated into a script tag unescaped)` `compiledTemplate = compiledTemplate.replace('<script id="subtitle"></script>', '<script id="subtitle" type="text/vtt" data-label="English" data-lang="en">' + subs + '</script>')`
Recommendation	HTML-escape the subtitle text and place it in a data attribute or safe element rather than inside a script tag. Files to change: routes/videoHandler.ts
Fix example	`const safe = escapeHtml(subs)`
Production risk of the fix	Subtitles containing intentional markup would render as text.
Regression test	Add a test with a subtitle of </script><script>alert(1)</script> and assert no script executes.
Classification	CWE-79 | A03:2021 Injection | CVSS 5.1 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:L/VI:L/VA:N/SC:L/SI:L/SA:N) | Confidence: Medium | Votes: 2 confirmed / 1 verify / 0 reject (dissent: One verifier could not find a request-reachable path for an attacker to set the subtitle file content.)

4. Findings Requiring Manual Verification

The items below are specific, source-grounded concerns that depend on one fact the repository does not show (for example a cloud, hosting, identity-provider or production setting). They are listed separately and must not be treated as confirmed vulnerabilities until verified.

V-06 Unsigned (alg=none) JWTs are accepted because the token is decoded without verifying the signature [UNVERIFIED]

Location	routes/verify.ts:119

Description	Juice Shop's own jwtUnsignedChallenge indicates an alg=none / unsigned-token acceptance weakness, and the code does use unverified decode helpers (lib/insecurity.ts:56; routes/verify.ts:119). However the cited lines are challenge scorekeeping, and the primary authorization paths call jws.verify / express-jwt against the public key. Whether an unsigned token is actually accepted for authentication hinges on the behaviour of the end-of-life express-jwt 0.1.3 dependency, which source alone does not settle.
Missing fact	Whether the express-jwt 0.1.3 middleware (and the jws.verify path) accepts a token with alg=none / an empty signature for a real authenticated request.
Why it needs verification	The cited decode calls are telemetry; the signature-verifying middleware is what gates auth. Confirming the bypass requires knowing how express-jwt 0.1.3 treats alg=none, which is library-version behaviour not visible in this repo.
Potential impact if confirmed	verify.ts decodes the token without checking the signature, so a token with alg=none and a chosen payload is accepted
Safe check to settle it	In a disposable local instance, send a request with a token whose header is {"alg":"none"} and a forged payload; observe whether it is accepted as authenticated. Do not run against production.

Evidence (source trace)	`1. routes/verify.ts:119  (jwt.decode does not verify the signature)`
	`  const decoded = jws.decode(token) ? jwt.decode(token) : null`
	`2. lib/insecurity.ts:56  (unverified decode helper)`
	`  export const decode = (token: string) => { return jws.decode(token)?.payload }`

5. Dependency & Secret Hygiene

Known-vulnerable packages found in manifests and lockfiles. Unless a finding above shows the vulnerable code is reachable, these are listed as present but not shown reachable. Upgrading is still recommended.

No known-vulnerable dependencies were reported by trivy. OSV-Scanner was skipped because no lockfile was found. Commit a lockfile (package-lock.json, yarn.lock, pnpm-lock.yaml, poetry.lock, requirements.txt, go.sum, Gemfile.lock, etc.) and re-run for full dependency-vulnerability coverage.

Secrets to rotate

Location	Line	Type (value masked)	Where
data/static/users.yml	88	Detected a Generic API Key, potentially exposing access to various services and sensitive operations. (redacted)	Current files
data/static/users.yml	296	Detected a Generic API Key, potentially exposing access to various services and sensitive operations. (redacted)	Current files

Location	Line	Type (value masked)	Where
frontend/src/app/faucet/faucet.component.ts	34	Detected a Generic API Key, potentially exposing access to various services and sensitive operations. (redacted)	Current files
frontend/src/app/manage-passkeys/passkey-providers.ts	9	Detected a Generic API Key, potentially exposing access to various services and sensitive operations. (redacted)	Current files
frontend/src/app/manage-passkeys/passkey-providers.ts	11	Detected a Generic API Key, potentially exposing access to various services and sensitive operations. (redacted)	Current files
frontend/src/app/manage-passkeys/passkey-providers.ts	20	Detected a Generic API Key, potentially exposing access to various services and sensitive operations. (redacted)	Current files
infrastructure/terraform/networking.tf	171	Identified a Private Key, which may compromise cryptographic security and sensitive data encryption. (redacted)	Current files
insecurity.ts	21	Asymmetric Private Key (redacted)	Current files
lib/insecurity.ts	21	Identified a Private Key, which may compromise cryptographic security and sensitive data encryption. (redacted)	Current files
networking.tf	171	Asymmetric Private Key (redacted)	Current files
routes/login.ts	64	Detected a Generic API Key, potentially exposing access to various services and sensitive operations. (redacted)	Current files
terraform/networking.tf	171	Identified a Private Key, which may compromise cryptographic security and sensitive data encryption. (redacted)	Current files

Rotate every credential listed, even if it was removed later: Git history keeps old values.

6. Coverage & What Was Not Checked

- Coverage units: 34 total (candidate: 29, covered: 2, deferred: 1, out_of_scope: 2)
- Scanner leads raised: 107 (traced by hunters: 0, dismissed with a reason: 0, dependency items: 0)

- Candidates verified: 29 (28 confirmed, 1 need verification, 0 rejected as false positives)

Not checked

- osv-scanner (known vulnerabilities in lockfiles) did not run: skipped
- data/static/codefixes / checklists/core-attack-classes.md: out_of_scope (Explicitly out of scope per engagement (challenge fix snippets).)
- frontend/src/assets/private / checklists/core-attack-classes.md: out_of_scope (Explicitly out of scope per engagement.)
- OSINT/steganography/timing/geo-stalking challenges / checklists/core-attack-classes.md: deferred (Source-only audit cannot settle runtime-only challenges (e.g. steganography, EXIF geo-stalking, timing, credential-guessing); listed as out of reach by design.)
- Runtime behaviour, live infrastructure, cloud console and hosting dashboard settings (source-only audit; see items requiring verification)

7. OWASP Mapping

OWASP Top 10:2025

Category	Confirmed	Needs verification	Status
A01:2025 Broken Access Control	-	-	No finding in checked scope
A02:2025 Security Misconfiguration	-	-	No finding in checked scope
A03:2025 Software Supply Chain Failures	-	-	No finding in checked scope
A04:2025 Cryptographic Failures	-	-	No finding in checked scope
A05:2025 Injection	-	-	No finding in checked scope
A06:2025 Insecure Design	-	-	No finding in checked scope
A07:2025 Authentication Failures	-	-	No finding in checked scope
A08:2025 Software or Data Integrity Failures	-	-	No finding in checked scope
A09:2025 Security Logging and Alerting Failures	-	-	No finding in checked scope
A10:2025 Mishandling of Exceptional Conditions	-	-	No finding in checked scope

OWASP API Security Top 10:2023

Category	Confirmed	Needs verification	Status
API3:2023 Broken Object Property Level Authorization	F-18, F-24	-	FAILED
API5:2023 Broken Function Level Authorization	F-19	-	FAILED
API8:2023 Security Misconfiguration	F-01	-	FAILED

OWASP Top 10 for LLM Applications 2025

Category	Confirmed	Needs verification	Status
LLM01:2025 Prompt Injection	F-21	-	FAILED

8. Priority Remediation Order

Order reflects severity and how findings enable each other. Items requiring verification are placed where they would rank if confirmed.

#	Finding	Severity	Action
1	F-05	CRITICAL	Load the signing key from a secret manager or environment variable, rotate the exposed key immediately, and upgrade jsonwebtoken to a supported release.
2	F-18	CRITICAL	Whitelist the registration fields and hard-set role to 'customer' on the server.
3	F-01	HIGH	Replace the template-literal query with a parameterised Sequelize query using replacements, and compare the password in the WHERE clause only via a bound value.
4	F-02	HIGH	Use a parameterised query with a bound LIKE value and escaped wildcards.
5	F-03	HIGH	Replace $where with a direct field match, e.g.
6	F-04	HIGH	Coerce id to a single ObjectId, remove multi:true, and verify the review's author matches the caller.
7	F-09	HIGH	Parse with entity expansion and DTD loading disabled and without filesystem input providers.
8	F-10	HIGH	Enforce a strict byte-size limit and use a YAML loader configuration that disallows/limits aliases before parsing.

#	Finding	Severity	Action
9	F-11	HIGH	Replace the eval with JSON.parse plus schema validation (e.g.
10	F-12	HIGH	Remove the eval, render the username as escaped text, and send a fixed strict CSP without 'unsafe-eval' or reflected values.
11	F-19	HIGH	Restore the authorization middleware (and role check) on the product update route.
12	F-23	HIGH	Reject registration when the credentialID belongs to another user, and move issuePasskeySession inside the verification.verified branch.
13	F-24	HIGH	Replace the dynamic field projection with a fixed allowlist of non-sensitive fields.
14	F-26	HIGH	Require current to be present and correct before changing the password;
15	V-06	UNVERIFIED	In a disposable local instance, send a request with a token whose header is {"alg":"none"} and a forged payload; observe whether it is accepted as authenticated.
16	F-07	MEDIUM	Replace MD5 with bcrypt or argon2id and rehash on successful login.
17	F-08	MEDIUM	Parse the candidate URL and require new URL(url).origin to exactly equal an allowlisted origin.
18	F-13	MEDIUM	Validate the URL scheme and resolved IP (reject private/link-local ranges), disable redirects, and verify the response content-type is an image.
19	F-14	MEDIUM	Remove the bypassSecurityTrust* calls and render values via normal interpolation (which escapes), or sanitize explicitly when HTML is required.
20	F-15	MEDIUM	Remove the serveIndex listing, delete backup files from ftp/, and serve only an explicit allowlist of files.
21	F-16	MEDIUM	Remove the /support/logs listing and route and keep logs outside any web-served path.
22	F-17	MEDIUM	Protect /metrics behind authentication or bind it to an internal interface/allowlist.
23	F-20	MEDIUM	Keep the client guards but enforce role/authorization server-side on every endpoint these pages call.
24	F-21	MEDIUM	Enforce the maximum discount in code before calling generateCoupon, independent of the model output.
25	F-22	MEDIUM	Reject requests whose decoded filename contains a null byte, and check the extension on the decoded, normalised name.
26	F-25	MEDIUM	Configure Express trust proxy and key the limiter on req.ip, ignoring raw X-Forwarded-For.

#	Finding	Severity	Action
27	F-27	MEDIUM	Upgrade sanitize-html, jsonwebtoken and express-jwt to maintained versions and re-test the sanitisation and token paths.
28	F-28	MEDIUM	Remove ngy-cookie, install the legitimate ngx-cookie (or an alternative), and audit the lockfile for other lookalikes.
29	F-29	MEDIUM	HTML-escape the subtitle text and place it in a data attribute or safe element rather than inside a script tag.

9. Disclosure

This report was prepared from an authorized, read-only review of the application's source code. No target code was executed and no live system, API, database or cloud account was contacted. No exploit code, live credential values or step-by-step exploitation sequences are included, in line with responsible disclosure practice. This document is confidential and intended solely for the teams responsible for remediation.

Findings reflect the source at the commit stated in the scope section. A clean result means no issue passed the evidence gate in the checked scope; it is not a guarantee that no vulnerability exists.

Generated with AI-Security-Audit-Skills v1.5.1 (https://github.com/risewithvj/AI-Security-Audit-Skills), created by Vijaya Kumar L (Rise With VJ).

Report Date: November 11, 2026